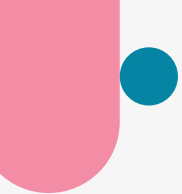


- **Software Qualität**
Schnittstellentechnologien:
gRPC- GraphQL

Kalil SACKO



Agenda

Motivation

GraphQL in detail

gRPC in detail

Live –Demo

Fazit



Motiv: warum wurden GraphQL und gRPC entwickelt

Warum wurde GraphQL entwickelt (von Facebook im Jahr 2012)?

Problem:

Die REST-APIs waren zu starr.

Auf Mobilgeräten lieferte REST entweder zu viele unnötige Daten oder nicht genug – dadurch waren mehrere Anfragen nötig.

Das führte zu **Over-Fetching**, **Under-Fetching** und Performance-Problemen.

Lösung (GraphQL):

Ermöglicht dem Client, **genau die Daten abzufragen**, die er benötigt – **nicht mehr und nicht weniger**.

Eine einzige Anfrage kann Daten aus mehreren Ressourcen aggregieren.

Reduzierung der Anzahl von HTTP-Anfragen.

Maximale Flexibilität für komplexe Benutzeroberflächen (Web & Mobile).

„GraphQL wurde entwickelt, um eine schnellere, flexiblere und präzisere Client-Erfahrung zu ermöglichen als REST.“

Warum wurde gRPC entwickelt (von Google im Jahr 2015)?

Problem:

Microservices müssen effizient miteinander kommunizieren. REST (HTTP/1.1 + JSON) ist **zu umfangreich, langsam** und **nicht für die Maschinen-zu-Maschinen-Kommunikation** optimiert.

Es bestand Bedarf an **Streaming**, **Authentifizierung**, **Mehrsprachigkeit**, usw.

Lösung (gRPC):

Verwendet **HTTP/2** → Multiplexing, Kompression.

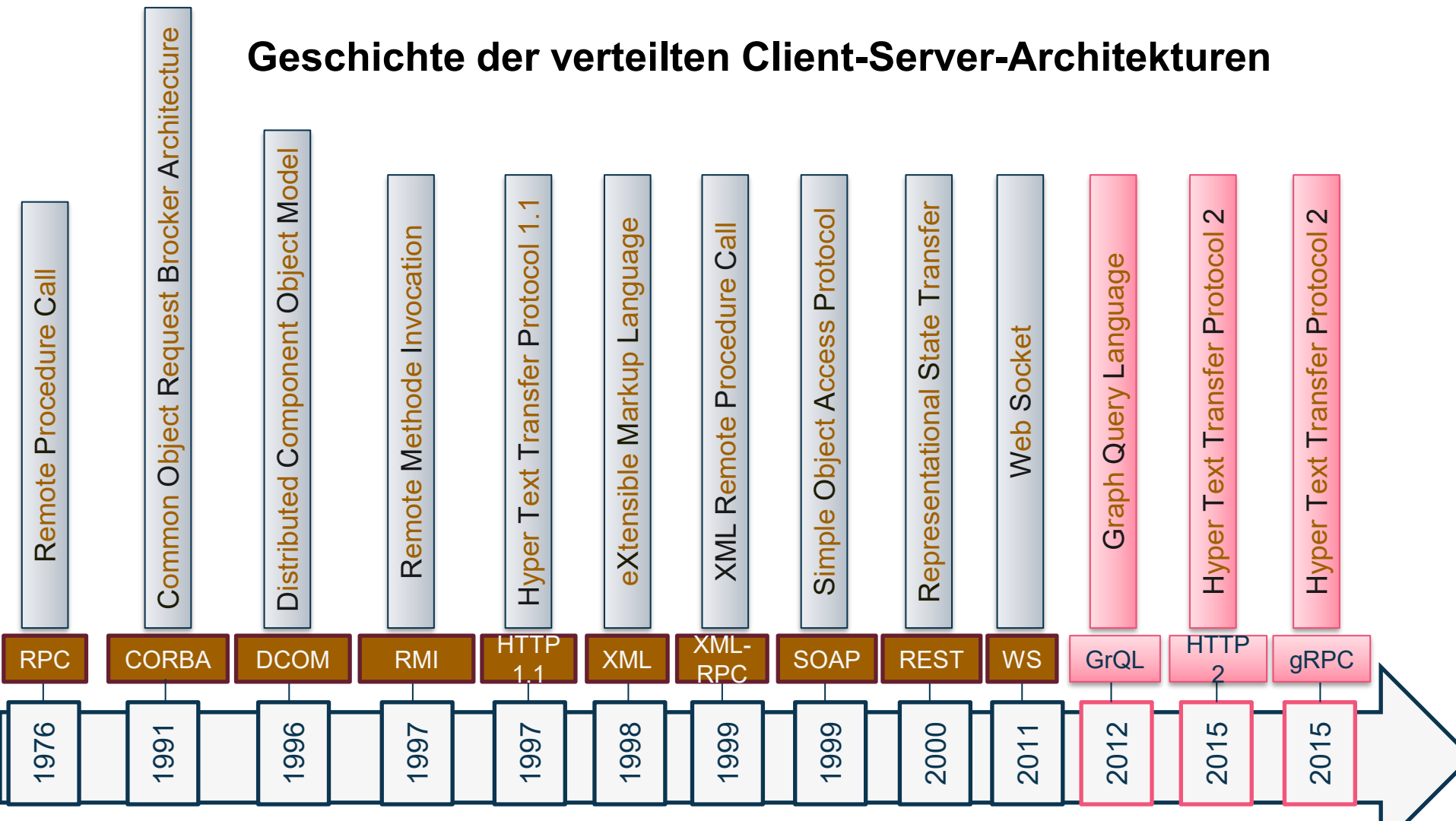
Verwendet **Protobuf** → Binärformat, sehr kompakt und schnell.

Unterstützt **Streaming** (Client → Server, Server → Client, bidirektional).

Generiert automatisch **Client- und Server-Code** in verschiedenen Programmiersprachen.

„gRPC wurde entwickelt, um eine ultraschnelle, effiziente und plattformübergreifende Kommunikation zwischen Microservices zu ermöglichen.“

Geschichte der verteilten Client-Server-Architekturen





GraphQL **?????**

GraphQL

GraphQL (kurz für *Graph Query Language*) ist eine Abfragesprache und eine Laufzeitumgebung, die 2012 von Facebook entwickelt und 2015 als Open-Source-Projekt veröffentlicht wurde. Innerhalb des Client-Server-Modells bietet GraphQL eine Alternative zu REST-APIs.

Eine **GraphQL-API** ist eine spezielle Art von API, die die Art und Weise revolutioniert, wie Clients Daten abrufen. Im Gegensatz zu traditionellen REST-APIs, bei denen jeder Endpunkt vordefinierte Daten zurückgibt, ermöglicht GraphQL den Clients, genau zu definieren, welche Daten sie benötigen – und diese in einem strukturierten Format zu erhalten.

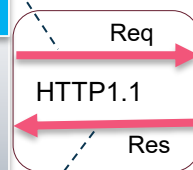
Dieser Ansatz bietet Entwicklern eine bisher unerreichte Flexibilität, da sie flexible Abfragen formulieren können, um exakt die benötigten Informationen zu erhalten, ohne unnötige Daten über das Netzwerk zu übertragen.

Die Besonderheit von **GraphQL** besteht darin, dass die Struktur der Serverantwort vom **Client** festgelegt wird (natürlich im Rahmen der API-Beschränkungen). Zum Beispiel entscheidet **der Client selbst**, welche Felder er für jedes Objekt erhalten möchte **und in welcher Reihenfolge** diese zurückgegeben werden sollen.

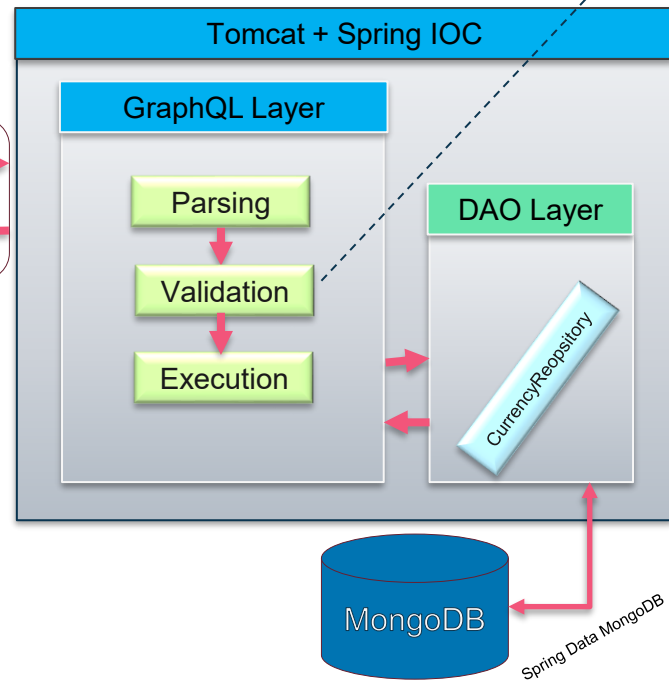
```
(Send mutation)
1 v mutation{
2   convertCurrency(convertRequest:{
3     currencyFrom:"eur",
4     currencyTo:"gnf",
5     amount:100
6   })
7 v {
8   currencyFrom,
9   currencyTo,
10  amount,
11  conversionResult
12 }
13 }
```

```
type Mutation{
  createAccount(createRequest : CreateAccountRequest): BankAccount
  updateAccount(accountId: String!, updateRequest: UpdateAccountRequest ) : BankAccount
  convertCurrency(convertRequest: ConvertCurrencyRequest): ConvertCurrencyResponse
}
```

```
input ConvertCurrencyRequest{
  currencyFrom: String
  currencyTo:String
  amount: Float
}
```



```
200 18ms
1 v {
2 v  "data": {
3 v    "convertCurrency": {
4     "currencyFrom": "EUR",
5     "currencyTo": "GNF",
6     "amount": 100,
7     "conversionResult": "908333.3333333334"
8   }
9 }
10 }
```



GraphQL-Schema

Alles beginnt mit der Definition eines GraphQL-Schemas. Das GraphQL-Schema beschreibt die verfügbaren **Datentypen** sowie die **Beziehungen** zwischen ihnen. Es handelt sich um eine **einheitliche Darstellung** aller Funktionalitäten der API. Das Schema wird mit einer **schemaspezifischen Sprache von GraphQL** definiert – dem sogenannten **GraphQL Schema Language (GQL)**.

```
73
74 #===== Callable Queries and Mutations from client=====
75 type Query{
76     getAccount(accountId:String!): BankAccount
77     getListAccounts: [BankAccount]
78     getListTransactions:[Transaction]
79 }
80
81
82 type Mutation{
83     createAccount(createRequest : CreateAccountRequest): BankAccount
84     updateAccount(accountId: String!, updateRequest: UpdateAccountRequest ) : BankAccount
85     convertCurrency(convertRequest: ConvertCurrencyRequest): ConvertCurrencyResponse
86 }
```

```
31
32 type Currency{
33     id:String
34     name: String!
35     symbol: String!
36     price : Float!
37 }
38
39 type Transaction{
40     id: String
41     accountId: String!
42     timestamp:Float
43     amount : Float!
44     type: TransactionType!
45     status: TransactionStatus!
46 }
47
48 #=====Mutation objects=====
49 input CreateAccountRequest{
50     balance: Float
51     type : AccountType!
52     state : AccountState!
53     currencySymbol: String!
54 }
55 input UpdateAccountRequest {
56     #accountId: String!
57     type : AccountType
58     state : AccountState
59 }
```

```
1 enum AccountType{
2     CURRENT_ACCOUNT
3     SAVING_ACCOUNT
4 }
5 enum AccountState{
6     CREATED
7     ACTIVATED
8     SUSPENDED
9     BLOCKED
10 }
11 enum TransactionType{
12     DEBIT
13     CREDIT
14 }
15 enum TransactionStatus{
16     PENDING
17     EXECUTED
18     REJECTED
19 }
20 #=====Base Object=====
21 type BankAccount{
22     id: String
23     balance : Float!
24     createdAt : Float
25     type : AccountType!
26     status : AccountState!
27     currency: Currency!
28
29 }
```

GraphQL-Resolvers

Sobald das Schema definiert ist, werden sogenannte **Resolver** eingerichtet.

Resolver sind Funktionen, die die **Daten für jedes Feld eines Datentyps abrufen**.

Zum Beispiel: Um die Daten eines Feldes „**price**“ des Typs „**Currency**“ zu erhalten, wird ein Resolver eingerichtet, der die **Datenbank abfragt**, um den mit diesem **Currency** verknüpften **price** abzurufen

```
@QueryMapping
public List<Account> getListAccounts() {
    return accountRepository.findAll();
}

@QueryMapping
public List<Transaction> getListTransactions(){
    List<AccountTransaction> accountTransactions = accountTransactionRepository.findAll();
    List<Transaction> transactions = accountTransactions.stream().map(
        AccountTransaction accountTransaction -> transactionMapper.fromEntityTransaction(accountTransaction) )
        .collect(Collectors.toList());

    return transactions;
}
```

```
type Currency{
    id:String
    name: String!
    symbol: String!
    price : Float!
}
```

```
//A resolver for only the field price
@SchemaMapping(typeName = "Currency", field = "price")
public double resolvePrice(CurrencyDTO currencyDTO) {
    Optional<Currency> currency = currencyRepository.findById(currencyDTO.getId());
    return currency.map(Currency::getPrice).orElse( other: 0.0);
}

@QueryMapping
/* the fonction returns a currency without price. but the price will be retrieved by
resolvePrice function, because the output type of graphql is defined as a Currency
with the four attribute : name, symbol, id and price*/
public CurrencyDTO getResolvedCurrency(@ Argument String id) {
    Optional<Currency> currency = currencyRepository.findById(id);
    CurrencyDTO currencyDTO = CurrencyDTO.builder()
        .symbol(currency.get().getSymbol())
        .name(currency.get().getName())
        .id(currency.get().getId())
        .build();
    return currencyDTO;
}
```

```
#===== Callable Queries and Mutations from client=====
type Query{
    getAccount(accountId:String!): BankAccount
    getListAccounts: [BankAccount]
    getListTransactions:[Transaction]
    getResolvedCurrency(id:String): Currency
}
```

Query and Mutation

Anfragen und Mutationen:

GraphQL-Clients können Anfragen stellen, um Daten von der API abzurufen. Diese Anfragen werden mit der **GraphQL-Abfragesprache** formuliert, die es den Clients ermöglicht, genau die Daten anzugeben, die sie benötigen. Die Anfragen können auch Argumente enthalten, um die Ergebnisse zu filtern, zu sortieren oder zu paginieren. Neben Anfragen können Clients auch Mutationen durchführen, um Daten auf der Serverseite zu erstellen, zu aktualisieren oder zu löschen.

Allgemeine Struktur einer GraphQL-Anfrage:

Eine GraphQL-Anfrage besteht aus vier Hauptbestandteilen:

- [1] **Operationstyp** (query, mutation)
- [2] **Name der Operation** (z. B. *getListTransaactions*)
- [3] **Argumente falls es gibt** (z. B. *id*, *input*)
- [4] **Projektion** (die Felder, die man als Antwort erhalten möchte)

```
► (Send mutation)
1 v mutation {
2   createAccount(createRequest: {
3     balance: 100.0,
4     type: SAVING_ACCOUNT,
5     state: ACTIVATED,
6     currencySymbol: "gnf"
7   }) {
8     id,
9     balance,
10    type,
11    status,
12    currency {
13      name,
14      price,
15    }
16  }
17 }
```

```
► (Send query)
1 v query{
2   getListTransactions
3 v {
4     id,
5     accountId,
6     amount,
7     timestamp,
8     type,
9     status
10  }
11 }
```

```
► (Send mutation)
1 v mutation{
2   convertCurrency(convertRequest:{
3     currencyFrom:"eur",
4     currencyTo:"gnf",
5     amount:100
6   })
7 v {
8     currencyFrom,
9     currencyTo,
10    amount,
11    conversionResult
12  }
13 }
```

Vor- und Nachteile

Vorteile von GraphQL

1. Flexibilität der Abfragen

Der Client wählt genau die Felder aus, die er abrufen möchte – nicht mehr und nicht weniger.

2. Reduzierung von Over-Fetching

Weniger unnötige Daten werden übertragen im Vergleich zu REST, das oft komplette Strukturen zurückgibt.

3. Einzelner Endpunkt (Single Endpoint)

Nur eine einzige URL für alle Anfragen – es ist nicht nötig, mehrere Routen wie bei REST zu definieren.

4. Introspektion

GraphQL erlaubt es, automatisch die Struktur der verfügbaren Daten im API zu entdecken.

5. Aggregation aus mehreren Quellen

Ermöglicht das Kombinieren von Daten aus mehreren Datenbanken oder APIs in einer einzigen Antwort.

Nachteile von GraphQL

1. Komplexität auf Serverseite

Man muss für jedes Feld oder jeden Typ eigene Resolver entwickeln.

2. Weniger geeignet für Caching

Da jede Anfrage unterschiedlich sein kann, ist das Caching auf Serverseite schwieriger umzusetzen.

3. Kann sehr wortreich sein (Verbose)

Die Abfragen können lang und schwer lesbar werden, besonders bei vielen verschachtelten Feldern.

4. Anspruchsvollere Sicherheit

Es muss genau gefiltert und validiert werden, was der Client abfragen darf.

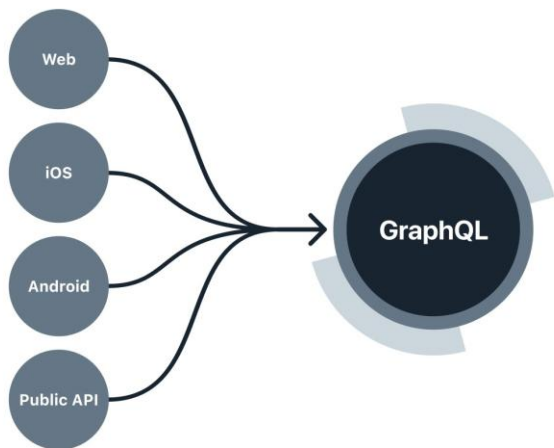
5. Höhere Lernkurve

Das Konzept von Schema, Resolvieren usw. ist neu für Entwickler, die an REST gewöhnt sind.

Fallstudien von GraphQL

twitter

Twitter properties



Twitter verwendet GraphQL, um den Datenzugriff über alle seine Plattformen hinweg zu vereinheitlichen (Web, iOS, Android und öffentliche API):

Ursprüngliches Problem: Die alte API v1.1 basierte auf zahlreichen REST-Mikrodiensten, was zu einem fragmentierten und schwer wartbaren System führte.

Lösung mit GraphQL: Twitter entwickelte eine einheitliche Datenschicht mit GraphQL, die sowohl von internen Clients (Twitter-Apps) als auch von der öffentlichen API genutzt wird.

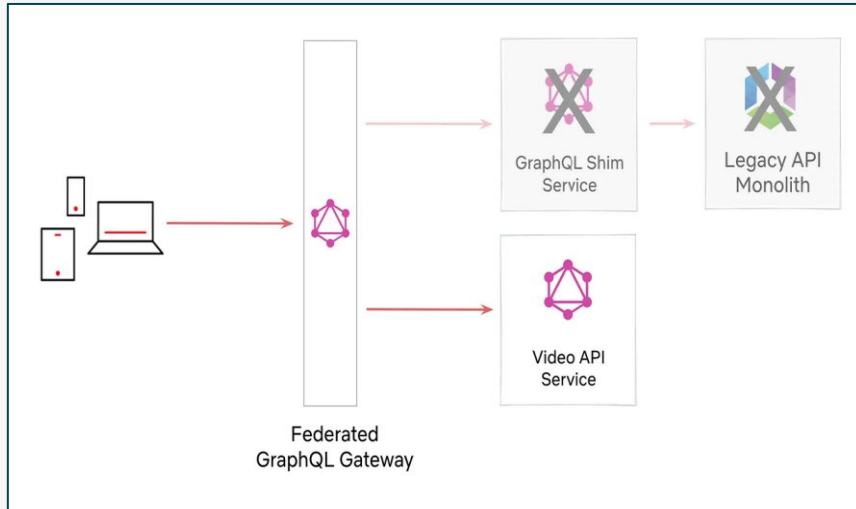
Hauptvorteil: Dieser Ansatz ermöglicht es den Teams, deklarativ auf Daten zuzugreifen, ohne sich mit der komplexen Logik von HTTP-Services auseinandersetzen zu müssen. Außerdem wird das Einführen neuer Funktionen erleichtert, da die Daten bereits über GraphQL verfügbar sind.

Ergebnis: Mehr Konsistenz in der API, weniger spezifische Logik pro Endpoint und ein System, das leichter weiterentwickelt werden kann.

https://blog.x.com/engineering/en_us/topics/infrastructure/2020/rebuild_twitter_public_api_2020

Fallstudien von GraphQL

Netflix



<https://hygraph.com/blog/products-using-graphql>

Netflix – Revolutionierung der Inhaltsbereitstellung mit GraphQL

Ausgangssituation (2018):

Netflix nutzte eine interne REST-API für eine App namens *Monet*. Die Entwickler entschieden sich für **GraphQL**, um die Datenabfrage aus verschiedenen Quellen zu verbessern.

Warum GraphQL?

- Um **Overfetching** (das Abrufen unnötiger Daten) zu vermeiden,
- Die **Bandbreite** zu reduzieren,
- Nur die wirklich **benötigten Daten** mit gezielten Anfragen abzurufen.

Umstellung im Jahr 2022:

Netflix migrierte seine **mobilen iOS-/Android-Apps** auf **föderiertes GraphQL**, ohne Ausfallzeiten, für **Hunderte Millionen Nutzer**.

- Dabei wurde das alte interne Framework **Falcor** ersetzt.
- Jedes **Domänenteam** kann nun **eigenständig** seinen API-Bereich verwalten – dank der föderierten Architektur.

Wichtige Vorteile:

- ✓ Effizientere Datenabfragen
- ✓ Weniger Netzwerklast
- ✓ Bessere Skalierbarkeit durch eine verteilte API-Architektur
- ✓ Reibungslose Migration **ohne Downtime**



gRPC

?????



GRPC : GOOGLE REMOTE PROCEDURE CALL

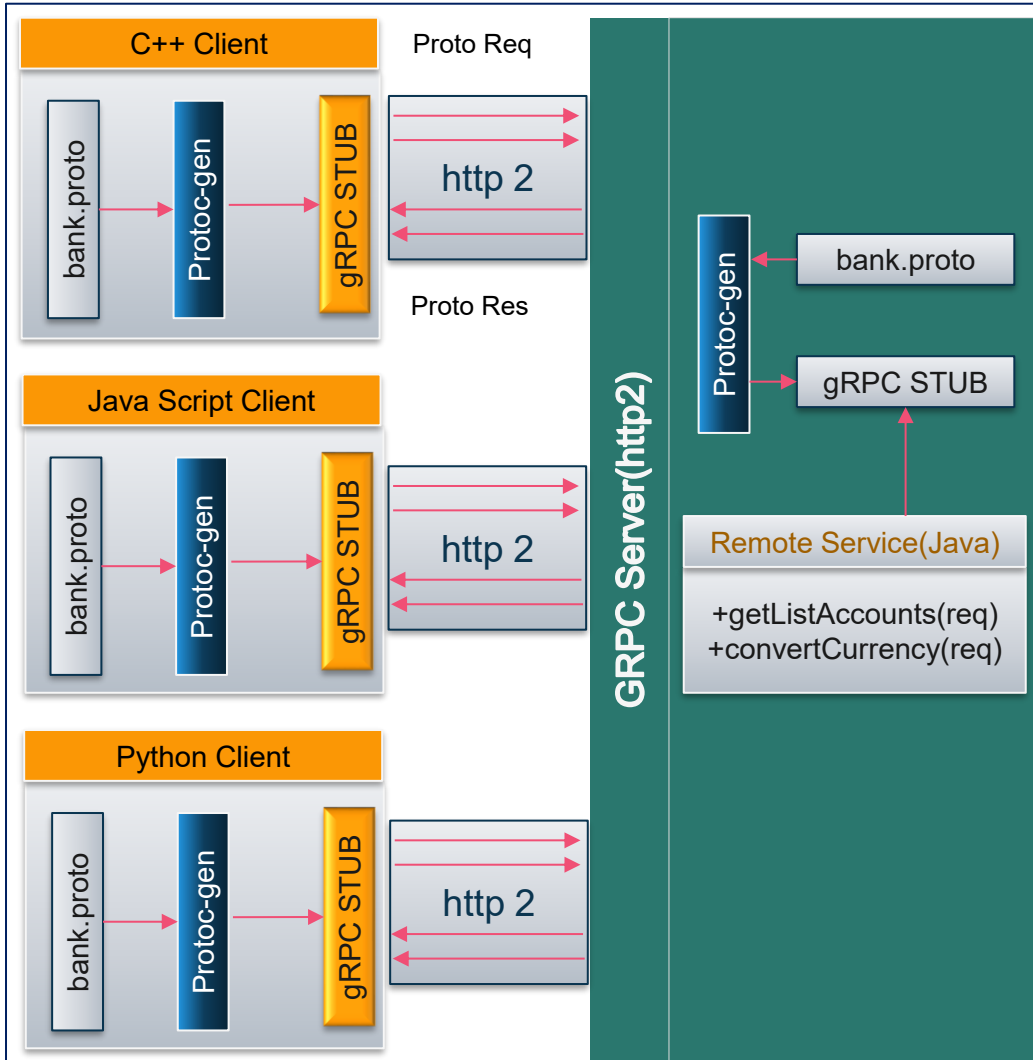
gRPC ist ein Framework, das es ermöglicht, die Funktionalitäten eines Dienstes für einen Fernzugriff über **HTTP/2** bereitzustellen.

Es erfüllt denselben Zweck wie seine Vorgänger: **RPC**, **CORBA**, **RMI**, **SOAP**, **REST** und **GraphQL**.

Ähnlich wie **CORBA** ermöglicht **gRPC** die Kommunikation zwischen mehreren verteilten Anwendungen, die verschiedene Programmiersprachen und Plattformen verwenden.

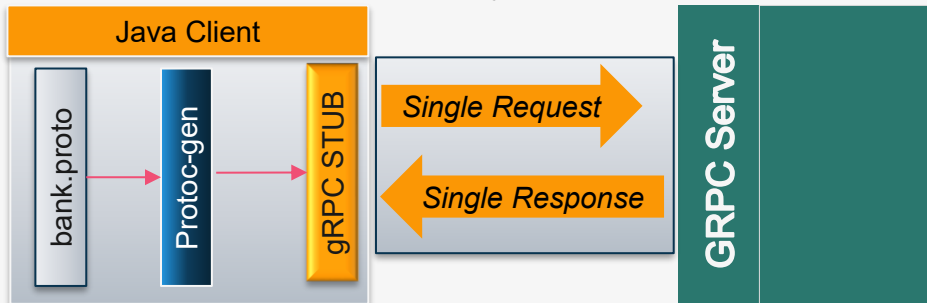
Es verwendet Protocol Buffers (**ProtoBuff**) als bevorzugten Serialisierungsmechanismus – entwickelt von Google als leistungsfähigere Alternative zu **IDL**, **XML** und **JSON**.
gRPC profitiert von den Vorteilen des **HTTP/2-Protokolls**, insbesondere:

- Bidirektionale Kommunikation (**Full Duplex**) zwischen Client und Server, was die Entwicklung reaktiver verteilter Anwendungen ermöglicht.
- Dauerhafte Verbindung während der gesamten Kommunikation (mehrere Anfragen können gleichzeitig über eine einzige **TCP-Verbindung** gesendet werden).
- Optimierung des ausgetauschten Inhalts.
- Client und Server können Daten kontinuierlich übertragen, was den Bedarf an neuen Anfragen oder zusätzlichen Verbindungen reduziert.

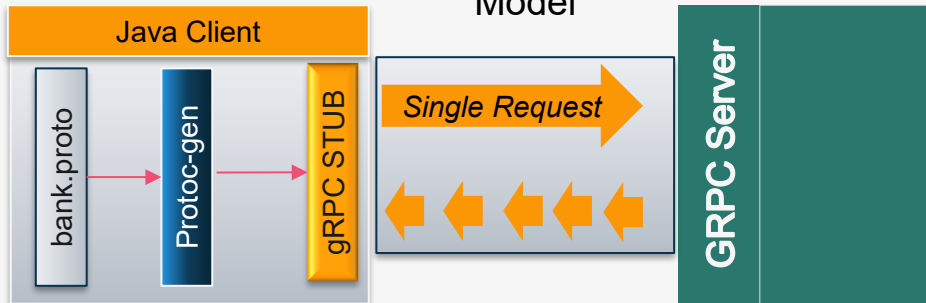


Kommunikationsmodelle mit gRPC: 4 Modelle

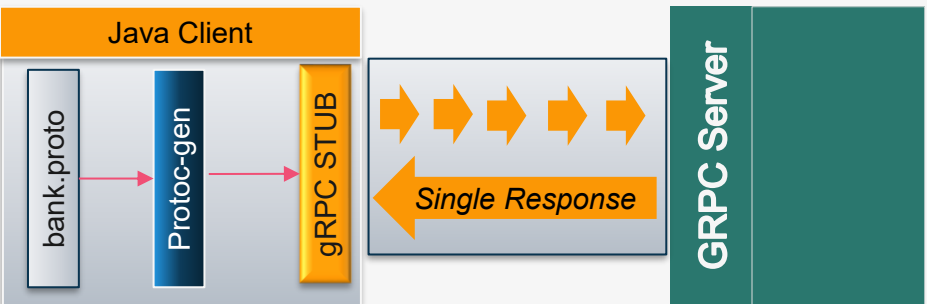
Unary Model



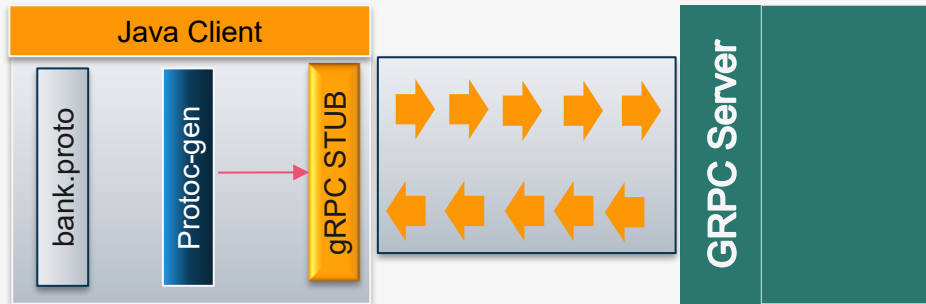
Server Streaming Model



Client Streaming Model



Bi-Directional Streaming Model



Protocol Buffer

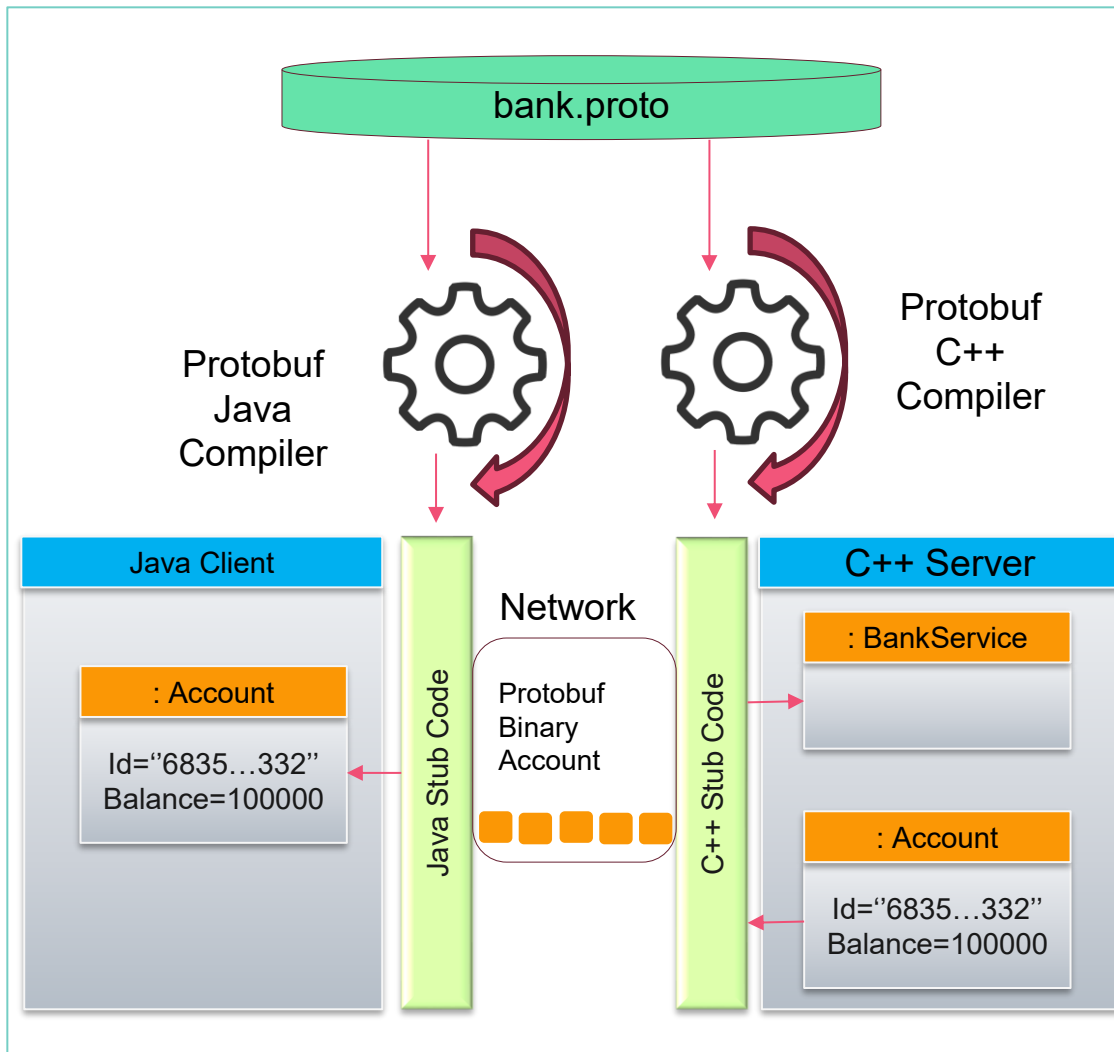
Protocol Buffers oder **Protobuf** ist ein **Serialisierungsformat** mit einer Beschreibungssprache für Schnittstellen, die von **Google** entwickelt wurde. Im Gegensatz zu **XML** und **JSON** ist der Inhalt einer Protobuf-Nachricht für den Menschen **nicht lesbar**, da die Nachricht im **Binärformat serialisiert** wird.

Die Größe einer serialisierten Nachricht in **Protobuf** ist **60 % kleiner** als bei **JSON**.

Protobuf ist schneller beim Kodieren und Dekodieren als **JSON**.

Die Struktur der Nachrichten und Dienste wird in einer separaten **„.proto“-Datei** deklariert – ähnlich wie **WSDL** bei **SOAP-Webdiensten**, die **XML-Schemata (XSD)** verwenden.

Diese **„.proto“-Datei** wird anschließend kompiliert, um **Stub-Code** in verschiedenen Programmiersprachen zu erzeugen:
C++, Java, Python, C#, Dart, Go, Ruby, Kotlin, Objective-C.

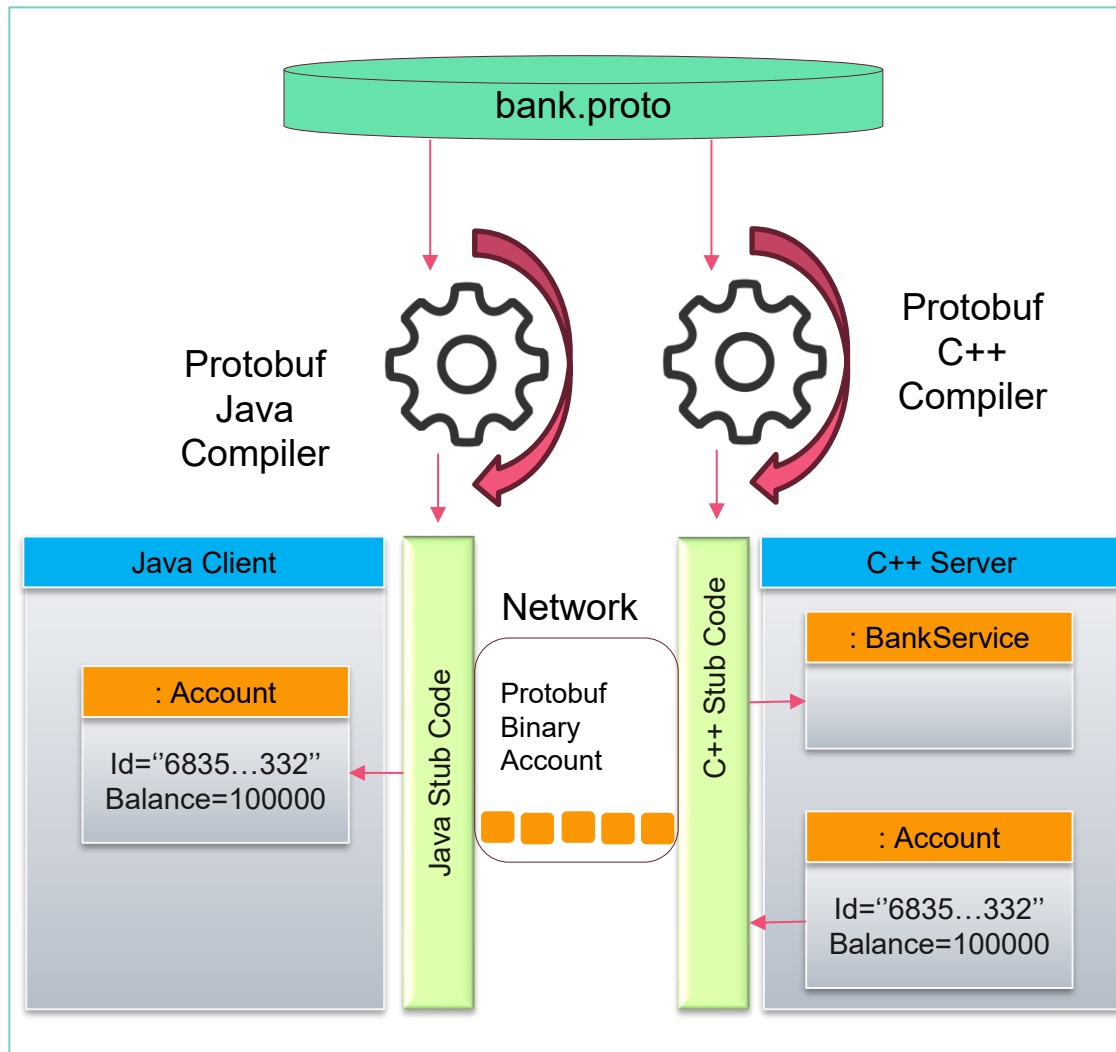


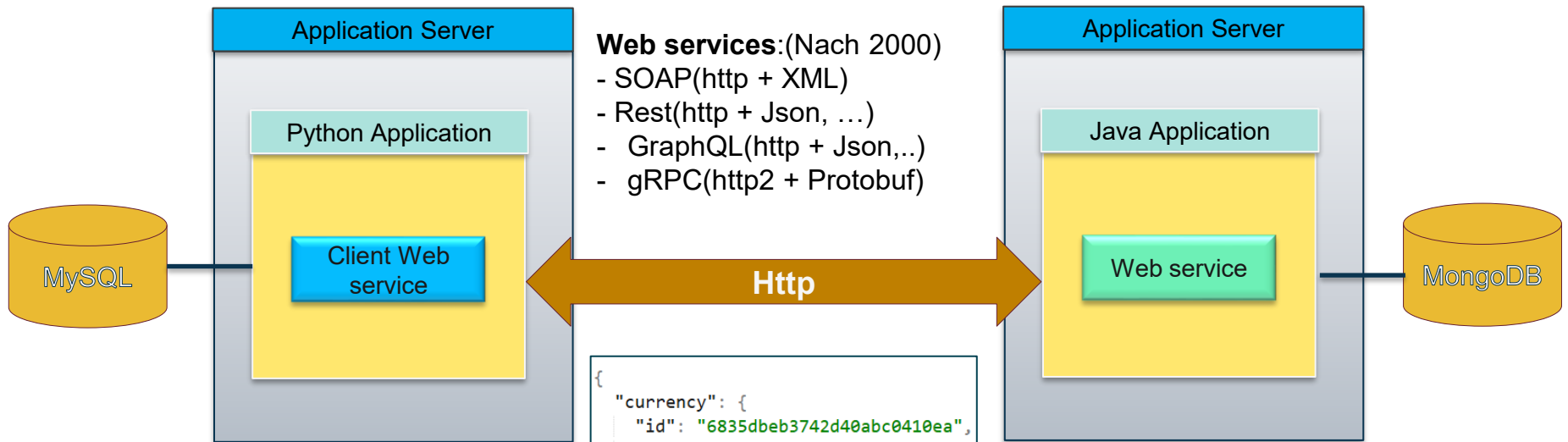
Protocol Buffer: Ziele

Ziele von Protobuf:

Verteilte Anwendungen **auf sehr performante Weise** miteinander kommunizieren lassen.

Ein **leistungsfähigeres Modell** zum Kodieren und Dekodieren bieten als bestehende Lösungen wie XML-Parser oder JSON-Serialisierungswerkzeuge.





bank.proto(GRPC)

```
message Currency{
  string id=1;
  string name=2;
  string symbol=3;
  double price=4;
}
```

```
{
  "currency": {
    "id": "6835dbeb3742d40abc0410ea",
    "name": "GUINEAN FRANC",
    "symbol": "GNF",
    "price": 0.00012
  }
}
```

132 B

```
<?xml version="1.0" encoding="UTF-8" ?>
<currency>
  <id>6835dbeb3742d40abc0410ea</id>
  <name>GUINEAN FRANC</name>
  <symbol>GNF</symbol>
  <price>0.00012</price>
</currency>
```

183 B

ID	Name	Symbol	Price
6835dbeb3742d40abc0410ea	GUINEAN FRANC	GNF	0,00012

Protobuf

1:'6835dbeb3742d40abc0410ea' 2: 'GUINEAN FRANC' 3: 'GNF' 00:04

Protofile: bank.proto

```
23 }
24 message BankAccount{
25     string accountId=1;
26     double balance=2;
27     int64 createdAt=3;
28     AccountType type=4;
29     AccountState state=5;
30     Currency currency=6;
31
32 }
33
34 message Currency{
35     string id=1;
36     string name=2;
37     string symbol=3;
38     double price=4;
39 }
40
41 message Transaction{
42     string id=1;
43     string accountId=2;
44     int64 timestamp=3;
45     double amount=4;
46     TransactionType type=5;
47     TransactionStatus status=6;
48
49 }
```

```
//=====REMOTE CALLABLE SERVICES=====
```

```
service BankService{
```

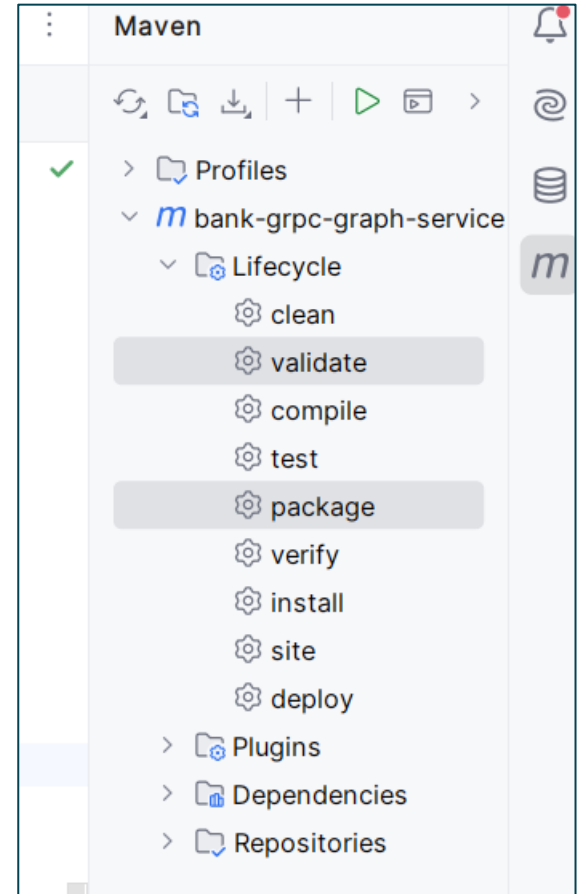
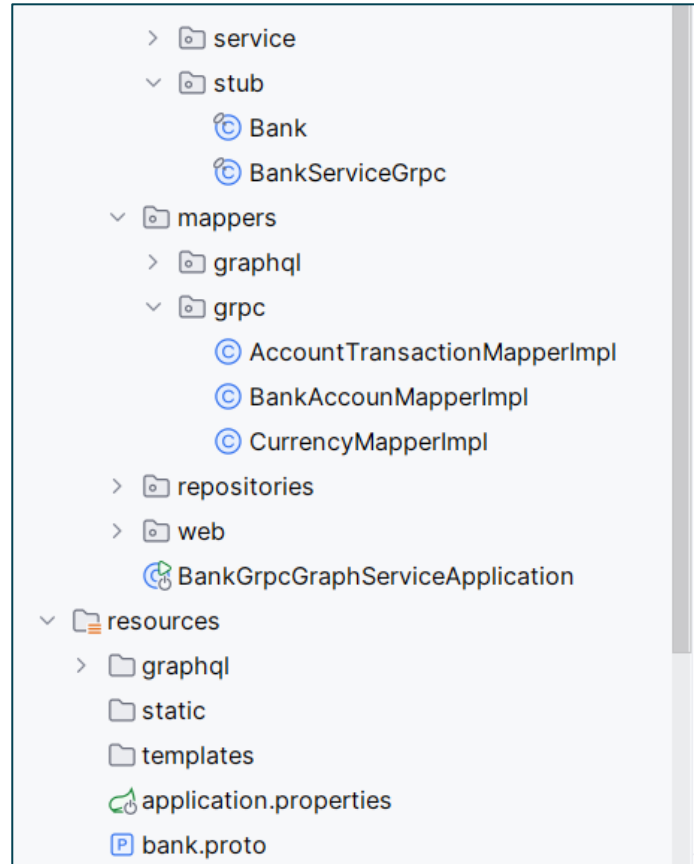
```
    //=====Unary model Services=====
```

```
    rpc createAccount(CreateAccountRequest) returns (BankAccount);
    rpc getAccount(GetAccountRequest) returns (GetAccountResponse);
    rpc getListAccounts(GetListAccountRequest) returns (GetListAccountResponse);

    rpc addCurrency(AddCurrencyRequest) returns (AddCurrencyResponse);
    rpc getCurrency(GetCurrencyRequest) returns (GetCurrencyResponse);
    rpc getListCurrencies(google.protobuf.Empty) returns (GetListCurrencyResponse);
    rpc convertCurrency(ConvertCurrencyRequest) returns (ConvertCurrencyResponse);
```

Kompilierung bank.proto

Nach der Kompilierung der .proto-Datei wird ein Verzeichnis namens stub generiert, das zwei zentrale Dateien enthält: **Bank** und **BankServiceGrpc**



Was ist in den Dateien enthalten (Bank und BankServiceGrpc)

Bank.java

Nach der Kompilierung der .proto-Datei erstellt der Protokoll-Compiler (protoc) automatisch eine Java-Datei mit dem gleichen Namen wie die ursprüngliche .proto-Datei – dabei wird der erste Buchstabe großgeschrieben.

Beispiel: bank.proto → Bank.java

Diese Datei enthält:

Alle Nachrichtendefinitionen (Messages), z. B.:

BankAccount

GetAccountRequest

GetAccountResponse

ConvertCurrencyRequest etc....

Alle Enumerationen (Enums), z. B.:

AccountType

AccountState

Diese Klassen sind typische POJOs (Plain Old Java Objects), die automatisch generiert werden. Sie beinhalten:

Getter- und Setter-Methoden

Konstruktoren

Hilfsmethoden zur Serialisierung/Deserialisierung von Nachrichten ins Binärformat

Methoden zur Umwandlung in JSON

Diese Datei bildet also das Herzstück der Datenstrukturen, mit denen Client und Server kommunizieren – ganz ohne manuelles Codieren!

Was ist in den Dateien enthalten(Bank und BankServiceGrpc)

BankServiceGrpc.java

Der Name dieser Datei stammt direkt vom Namen des Dienstes, den in der .proto-Datei definiert wurde.
Beispiel im .proto:

```
service BankService {  
  rpc getAccount(GetAccountRequest) returns (GetAccountResponse);  
  ...  
}
```

Der protoc-Compiler generiert daraus automatisch die Datei **BankServiceGrpc.java**, welche alle Klassen enthält, die notwendig sind, um:

- den Dienst als **Client** aufzurufen (Stub-Klassen),
- oder den Dienst auf der Serverseite bereitzustellen (Abstract-Base-Implementierung).

Vorbereitung der Demo

Wir möchten eine **Client-Server-Anwendung** auf Basis von **gRPC** und **GraphQL** erstellen, die einen Dienst bereitstellt, mit dem man Bankkonten abfragen kann.

Der Dienst bietet folgende Funktionalitäten:

Eine Währung von einer Devise in eine andere umrechnen

Ein Konto anhand seiner ID abrufen (Unary model für grpc)

Eine Liste von Konten abrufen

Einen Transaktionsfluss abrufen (Server streaming für grpc)

Einen Transaktionsfluss senden (Client streaming für grpc)

Einen Transaktionsfluss ausführen (bidirectional)

Ein **Konto** ist definiert durch:

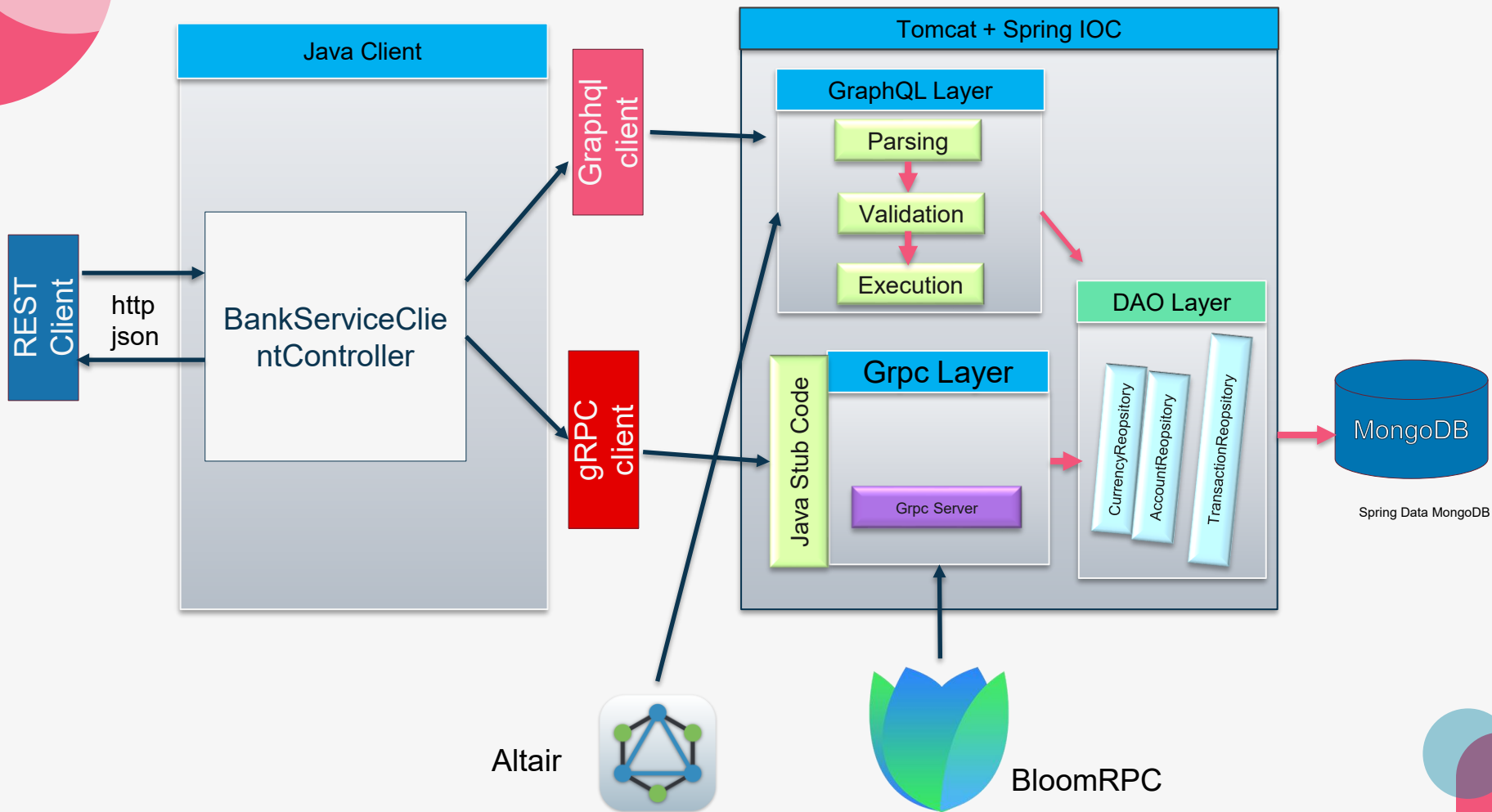
seine **ID**, sein **Erstellungsdatum**, sein **Kontostand**, seine **Währung**, seinen **Typ** (*Girokonto, Sparkonto*) und seinen **Status** (*aktiviert, suspendiert, ausstehend, geschlossen*).

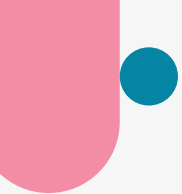
Eine **Transaktion** ist definiert durch:

ihre **ID**, das **Datum**, den **Betrag**, den **Typ** (*Lastschrift, Gutschrift*) und den **Status** (*ausstehend, ausgeführt, abgelehnt*).

Eine **Währungsumrechnungsanfrage** ist definiert durch den **Betrag**, die Ausgangswährung (**from**) und die Zielwährung (**to**)

Architecture





Live-Demo



Fazit

gRPC

Erstellt von: Google (2015)

Basierend auf: HTTP/2 + Protobuf (binäres Format)

Kommunikation: Sehr schnell, effizient, unterstützt

Streaming (Client-, Server- und bidirektionales Streaming)

Verwendet für: Kommunikation zwischen Microservices

oder zwischen Backend und Frontend in

leistungsintensiven Umgebungen

Anwendungsszenarien

Verteilte Systeme und Microservices

Echtzeitanwendungen (z. B. IoT, Chat, Streaming)

Kommunikation zwischen Diensten in mehrsprachigen

(polyglotten) Systemen

Interne APIs in großen Unternehmen

GraphQL

Erstellt von: Facebook (2012, Open-Source seit 2015)

Basierend auf: HTTP + JSON (Textbasiert)

Kommunikation: Flexibel, der Client bestimmt exakt, welche Daten er benötigt

(= keine Über- oder Unterabfrage)

Verwendet für: APIs zwischen Frontend und Backend,

besonders bei komplexen oder mobilen

Benutzeroberflächen

Anwendungsszenarien

Moderne Web- und Mobile-Apps mit dynamischen und flexiblen Datenanforderungen

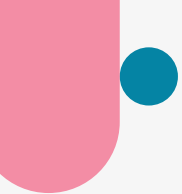
Aggregation von Daten aus mehreren Quellen in einer einzigen Anfrage

API-Gateways für Microservices

Developer-freundliche öffentliche APIs (z. B. GitHub, Shopify)

QUELLEN

<https://datascientest.com/apis-graphql-tout-savoir>,
https://blog.x.com/engineering/en_us/topics/infrastructure/2020/rebuild_twitter_public_api_2020
,
<https://hygraph.com/blog/products-using-graphql>
<https://en.wikipedia.org/wiki/GRPC>,
https://en.wikipedia.org/wiki/Web_service,
<https://en.wikipedia.org/wiki/SOAP>,
https://en.wikipedia.org/wiki/Common_Object_Request_Broker_Architecture,
<https://fr.wikipedia.org/wiki/GraphQL>



Vielen Dank

